

Queries explanation

Phase IV — Database Creation

Statement	Purpose
CREATE TABLESPACE fraud_db_ts	Dedicated storage area so this project's data is isolated from other schemas on the same Oracle instance.
CREATE USER Dianna_26585_2024_FraudDetection_DB	The schema owner, following the required naming convention.
GRANT CREATE SESSION / TABLE / VIEW / SEQUENCE / PROCEDURE / TRIGGER / SYNONYM	Minimum privileges needed to build and run everything in later phases — nothing more (least-privilege principle).
CREATE ROLE fraud_analyst_role	A read-only role, used later so Power BI or a junior analyst account can query data without having schema-owner rights.

Phase V — Table Implementation

Sequences (customer_seq, device_seq, etc.) generate surrogate primary keys automatically on insert, via DEFAULT <seq>.NEXTVAL on each table — so callers never have to supply an ID manually.

Tables and their constraints:

- **customer** — UNIQUE on national_id and phone_number prevents duplicate registrations; CHECK restricts kyc_status to a fixed set of values (PENDING/VERIFIED/REJECTED), so no typo can ever creep into that column.
- **device** — FOREIGN KEY (customer_id) ties every device to a real customer; UNIQUE on device_imei stops the same physical phone being registered twice.
- **agent** — standalone entity; UNIQUE on license_number prevents duplicate agent licenses.

- **account** — FOREIGN KEY (customer_id); CHECK (balance >= 0) makes it impossible for an account to go negative at the database level, regardless of what application code does.
- **transaction_** (named with a trailing underscore because TRANSACTION is an Oracle reserved word) — four foreign keys (sender account, receiver account, agent, device) tie a transaction to everything involved in it. CHECK (amount > 0) and CHECK (sender_account_id <> receiver_account_id) block two of the most common data-entry errors (zero/negative amounts, self-transfers) before they ever reach application logic.
- **fraud_rule** — the catalogue of detection rules (threshold, severity, active flag). Business users can add/deactivate rules here without touching PL/SQL code.
- **fraud_flag** — links a transaction_ to the fraud_rule that triggered it, with a risk_score (0–100, enforced by CHECK) and a resolution_status that tracks the case through its lifecycle.
- **analyst / investigation** — who is working a case and what they found. UNIQUE (flag_id) on investigation guarantees one investigation per flag (no duplicate case files).
- **public_holiday** — reference table for the mandatory weekday/holiday business rule enforced in Phase VII.
- **audit_log** — generic table capturing who changed what, when, and the before/after values, across any audited table.

Sample data was inserted deliberately to include one obvious fraud case (a 700,000 RWF transfer, account #2 → #3) so the PL/SQL and trigger logic in later phases has something real to catch during the live demo.

Phase VI — PL/SQL Programming

fn_calculate_risk_score(p_transaction_id) — function

Looks up a transaction, then adds points to a running score based on three independent signals: transaction amount (>500k = +50, >200k = +25), whether the device was first seen within the last day (+30, since brand-new devices are a common fraud vector), and whether the same device made more than 5 transactions in the last 10 minutes (+20, catching "smurfing"). The score is capped at 100. This logic is centralized in one function so every caller (the procedure, and potentially the trigger) scores transactions identically.

pr_register_transaction(...) — procedure

The single entry point for creating a new transaction. It:

1. Validates business rules first (sender $\neq$ receiver, amount > 0) using named exceptions — invalid data never reaches the INSERT.
2. Inserts the transaction and captures the generated ID via RETURNING ... INTO.
3. Calls fn_calculate_risk_score to score it.
4. Uses a **cursor** (c_active_rules) to loop through every active fraud rule and automatically create a fraud_flag if the transaction breaches a threshold.
5. Commits on success, rolls back and re-raises on any error — so a partially-completed transaction is never left in the database.

fraud_pkg — package

Groups all fraud-related operations behind one public interface, so calling code (or a future front-end) only needs to know the package, not four separate objects:

- register_transaction — wraps pr_register_transaction.
- open_investigation — validates the flag exists, inserts an investigation row, and moves the flag to UNDER_REVIEW.
- close_investigation — validates the outcome value, closes the investigation, and maps the outcome to the correct final fraud_flag status (CONFIRMED_FRAUD, FALSE_POSITIVE, or leaves it UNDER_REVIEW for INCONCLUSIVE).
- get_open_flag_count — a simple reporting function used to show the case backlog before/after processing.

Every procedure in the package **commits on success and rolls back on any exception**, and re-raises the error so the calling code (or the demo output) knows exactly what went wrong.

Phase VII — Triggers, Auditing & Security

trg_block_weekday_holiday_dml — compound trigger

Implements the brief's mandatory business rule: no INSERT/UPDATE/DELETE on transaction_ during weekdays or public holidays. It's a **compound trigger** using a single BEFORE STATEMENT section (not FOR EACH ROW) because the check — "what day is it today?" — is the same regardless of how many rows are affected, so there's no need to repeat it per row. It checks the current day name and cross-references public_holiday before allowing the statement to proceed.

trg_audit_account / trg_audit_fraud_flag — row-level audit triggers

Fire AFTER every INSERT/UPDATE/DELETE and write a row into audit_log capturing the operation type and the old/new values. Wrapped in EXCEPTION WHEN OTHERS THEN NULL deliberately — an audit failure should never block the real business transaction from completing.

trg_track_logon — system-level trigger

Fires AFTER LOGON ON SCHEMA, i.e. every time someone connects to this schema. Logs the username, OS user, and host machine into session_log — satisfies the brief's "user activity tracking" requirement independent of any specific table.

trg_protect_customer_delete — security trigger

An extra layer of security beyond the GRANT-based privileges from Phase IV: even someone with DELETE privilege on customer cannot actually delete a row unless they are the schema owner. Demonstrates defense-in-depth (privileges *and* trigger-level enforcement).